

VERBEX COMPOSITES S.L.

Arquitectura del agente

Guía técnica para desarrolladores

Track B · Python · Sesión II4 · COIIAOC 2026

verbex-agent_c · Bernardo Ronquillo Japón

0. Introducción	3
Cómo leer este documento.....	3
Paleta de colores del sistema (COIIAOC v1.1).....	3
1. El repositorio en una imagen — arquitectura modular.....	4
1.1 main.py como orquestador puro	5
1.2 La equivalencia n8n ↔ Python	5
1.3 Archivos de configuración.....	5
2. Flujo de control completo — process().....	6
2.1 Los cinco pasos de main.py.....	7
2.2 La bifurcación del duplicado	7
2.3 El fan-out y la deuda técnica hacia II5	7
3. Pipeline de reglas deterministas — R01–R09	8
3.1 El orden canónico y por qué importa	9
3.2 Validación dura vs. degradación suave	9
3.3 El principio E5 en acción	9
4. La capa cognitiva — llm.py	10
4.1 Fase 1 — Construcción del system prompt	11
4.2 Fase 2 — Llamada a la API.....	11
4.3 Fase 3 — Las 4 guardias del parser anti-alucinación	11
4.4 La frontera E5 en llm.py.....	12
5. Fan-out de notificaciones — notify.py.....	12
5.1 Los cuatro canales	13
5.2 La independencia de condiciones.....	13
5.3 Idempotencia asimétrica.....	13
6. Casos de prueba en clase — Gate F4	14
6.1 Los tres casos	14
6.2 Criterio Gate F4.....	14
7. Hoja de ruta hacia II5	15
7.1 Lo que queda fuera de scope en II4	15
7.2 Por qué la modularización no puede esperar más.....	15
7.3 Checklist pre-II5.....	15

0. Introducción

Este documento es el complemento técnico escrito de la sesión II4 del curso **"Automatización de Procesos con Agentes Inteligentes"** (COIIAOC, 2026). Está pensado para leerse **después de la sesión en clase**, como material de referencia que el alumno puede consultar al implementar el Track B en Python.

La sesión II4 construye el pipeline completo del agente VERBEX: desde el texto libre de una Purchase Order (PO) hasta sus cuatro canales de salida (Sheets, Telegram producción, Telegram calidad, email). El repositorio `verbex-agent_c` contiene el equivalente Python del workflow n8n desarrollado en el Track A.

Cómo leer este documento

Cada capítulo corresponde a un diagrama. La estructura es siempre la misma: primero el diagrama como imagen de referencia; después la explicación en prosa de lo que **no** es visible directamente en el código; y finalmente la conexión con los conceptos del curso (E5, PRDA, IO/RE).

Convención de nombres en código: los nombres de ficheros, funciones y variables aparecen en `monospace`. Los conceptos del curso (E5, PRDA, IO/RE) aparecen en **negrita**.

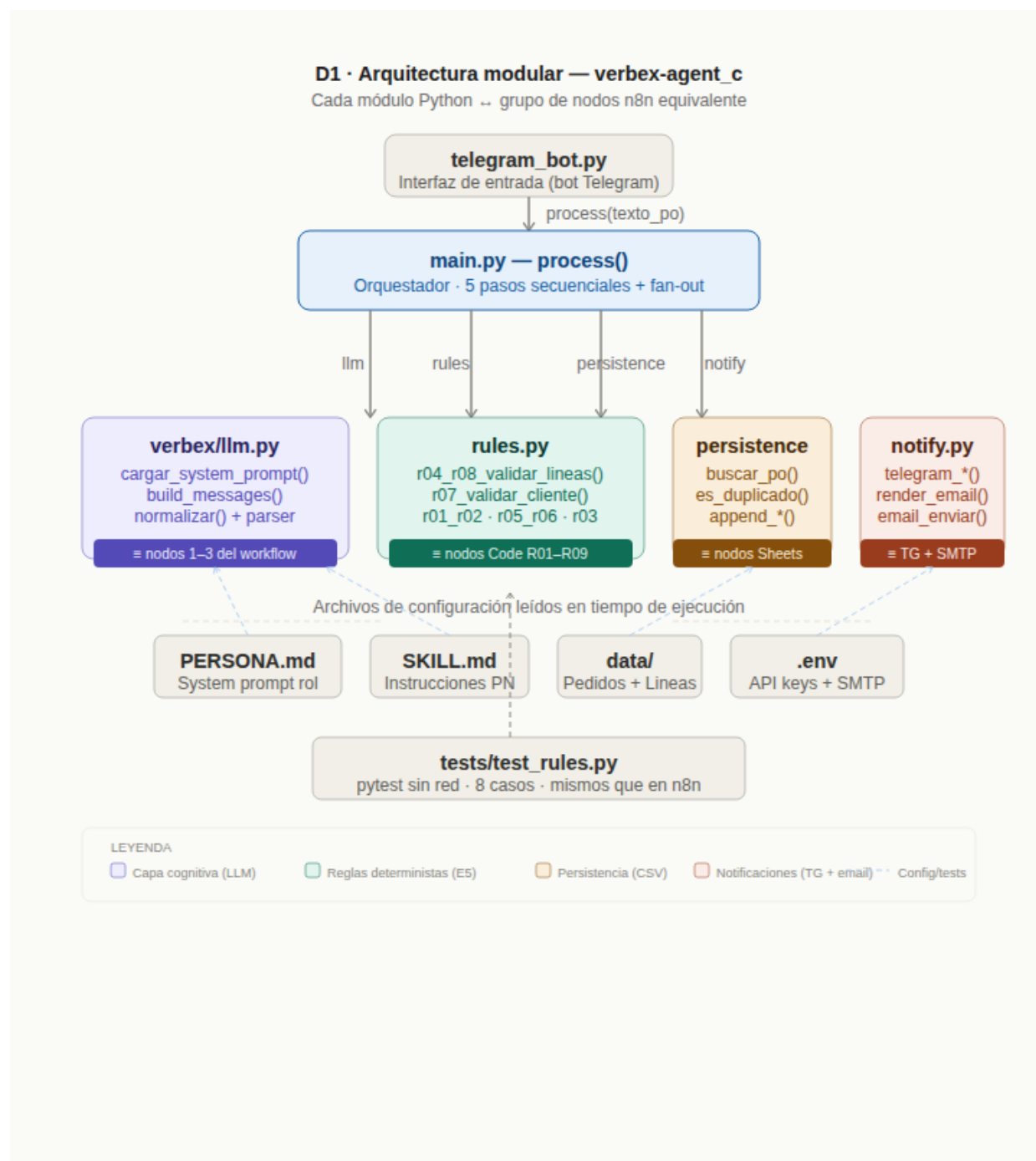
Paleta de colores del sistema (COIIAOC v1.1)

Todos los diagramas usan la misma paleta semántica. Aprender a leer los colores acelera la comprensión:

- **Morado** — capa cognitiva (LLM, prompt, parser anti-alucinación)
- **Verde** — capas deterministas: reglas E5, persistencia, outputs positivos
- **Ámbar** — lógica condicional: Switch de prioridad, degradaciones suaves (→ PARCIAL)
- **Rojo** — errores duros, validaciones fallidas, parada del flujo
- **Azul** — estructuras de datos de entrada, Filter de certificación
- **Gris** — nodos upstream/downstream, configuración, tests

1. El repositorio en una imagen — arquitectura modular

El primer paso para entender cualquier base de código es saber qué hace cada fichero y quién habla con quién. El diagrama D1 muestra esa imagen para `verbex-agent`



D1 — Arquitectura modular de `verbex-agent_c`

1.1 main.py como orquestador puro

El hallazgo más importante del diagrama: `main.py` no hace **nada** por sí mismo. No llama a la API de Claude, no escribe en CSV, no envía mensajes. Cada responsabilidad está delegada a un módulo con una frontera semántica precisa:

- `verbex/llm.py` — todo lo que involucra al LLM (prompt, llamada, parser)
- `rules.py` — todas las reglas deterministas R01–R09 (cero llamadas al LLM)
- `verbex/persistence.py` — lectura y escritura en los CSV locales
- `verbex/notify.py` — los cuatro canales de salida (Telegram × 2 + email)

Esta separación no es decorativa. Tiene una consecuencia práctica inmediata:

`tests/test_rules.py` puede ejecutarse sin red, sin credenciales y sin el LLM. Los 8 casos de prueba son exactamente los mismos que se validaron en `n8n` — lo que cambia es el contenedor, no el contrato.

1.2 La equivalencia `n8n` ↔ Python

Los badges de color en el diagrama indican a qué grupo de nodos `n8n` equivale cada módulo Python. La frase clave del curso aplica aquí directamente: **"solo cambia el contenedor"**. El system prompt que en `n8n` vivía pegado dentro del nodo Claude API ahora vive en `PERSONA.md` y `SKILL.md` — ficheros versionables, auditables, y legibles sin abrir `n8n`.

Punto de examen: ¿Por qué es una ventaja que el system prompt esté en ficheros separados en lugar de estar hardcodeado en el código Python?

1.3 Archivos de configuración

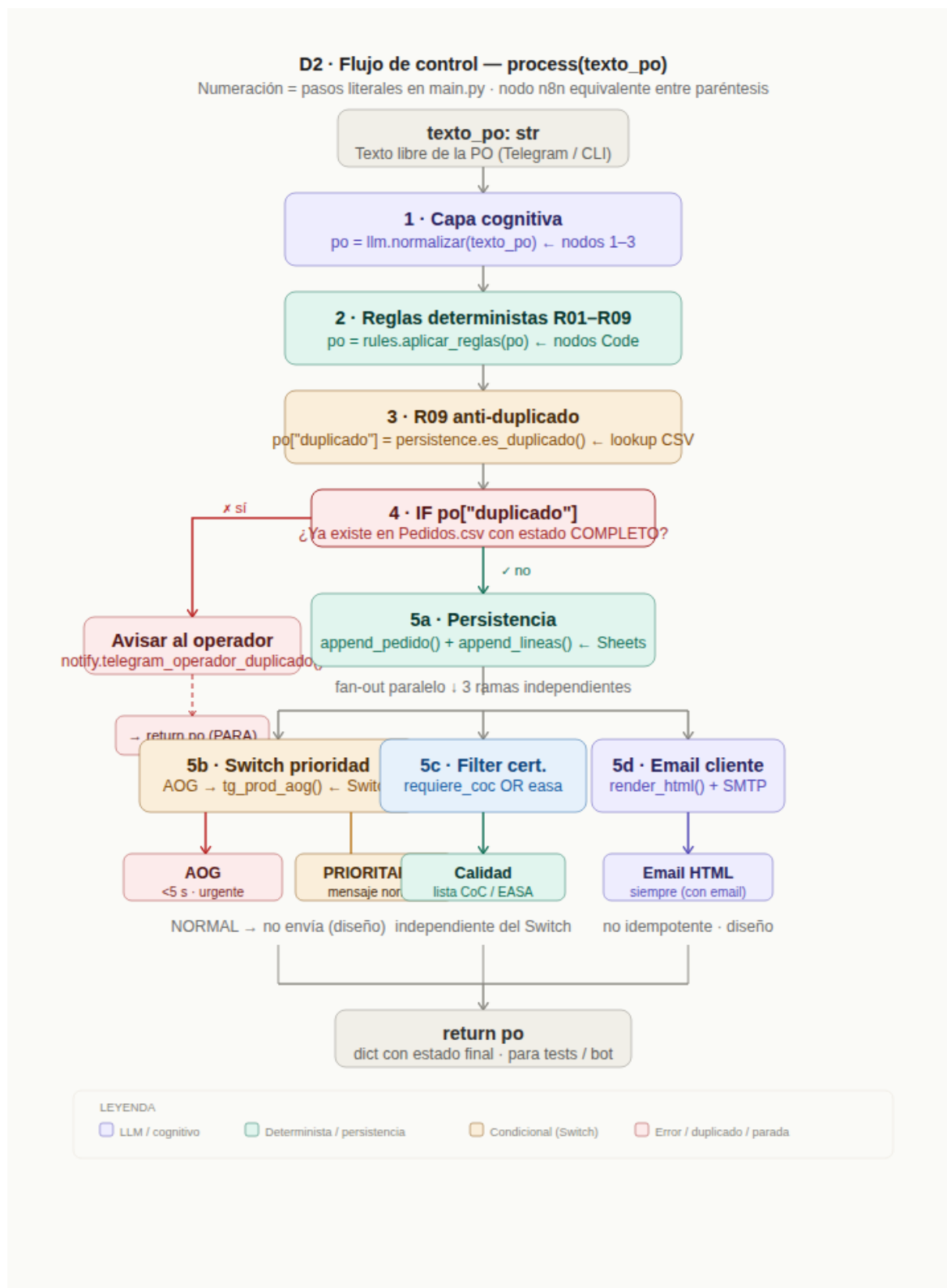
Las flechas punteadas azules del diagrama señalan dependencias de lectura en tiempo de ejecución, no importaciones de código. Cuatro ficheros alimentan al sistema sin ser módulos Python:

- `PERSONA.md` + `SKILL.md` → leídos por `llm.cargar_system_prompt()` al inicio de cada llamada
- `data/Pedidos.csv` + `data/Lineas.csv` → la memoria persistente del agente (equivalente a Google Sheets)
- `.env` → todas las credenciales (`ANTHROPIC_API_KEY`, `TELEGRAM_BOT_TOKEN`, `SMTP_PASSWORD`)

El fichero `.env` **nunca debe subirse al repositorio**. El `.gitignore` ya lo excluye, pero el alumno debe verificarlo antes de hacer cualquier `git push`.

2. Flujo de control completo — process()

Una vez entendida la estructura modular, el siguiente nivel es el flujo de control: qué ocurre exactamente cuando llega un mensaje de Telegram con el texto de una PO. Ese flujo está concentrado en la función `process()` de `main.py`, que actúa como el grafo del workflow n8n traducido a llamadas secuenciales y condicionales en Python.



D2 — Flujo de control completo de process()

2.1 Los cinco pasos de main.py

La función `process()` tiene exactamente cinco pasos, ejecutados siempre en el mismo orden. No hay lógica de routing entre ellos: cada paso recibe el dict `po` del paso anterior, lo enriquece, y lo pasa al siguiente.

1. `po = llm.normalizar(texto_po)` — texto libre → JSON estructurado (nodos 1–3 del workflow)
2. `po = rules.aplicar_reglas(po)` — aplicar R01–R09 sin tocar el LLM (nodos Code)
3. `po["duplicado"] = persistence.es_duplicado(...)` — lookup R09 real contra el CSV
4. IF duplicado → avisar al operador y **parar** — guardia anti-spam
5. Fan-out: persistencia + Switch + Filter + email — las cuatro ramas de salida

2.2 La bifurcación del duplicado

La rama izquierda del diagrama (el `if po.get("duplicado")`) es una **parada dura**: `return po` inmediato, sin tocar persistencia ni email. Esto es lo que hace al R09 realmente valioso — no es solo "no escribir dos veces en Sheets", es cortar **todo** el fan-out para evitar que el cliente reciba confirmaciones repetidas cuando Telegram reentrega el mismo mensaje.

¿Cuándo ocurre una reentrega de Telegram? El bot de Telegram puede reenviar el mismo update si el servidor tarda en responder o hay un timeout de red. Sin el R09, cada reentrega generaría una fila nueva en Sheets y un email adicional al cliente.

2.3 El fan-out y la deuda técnica hacia II5

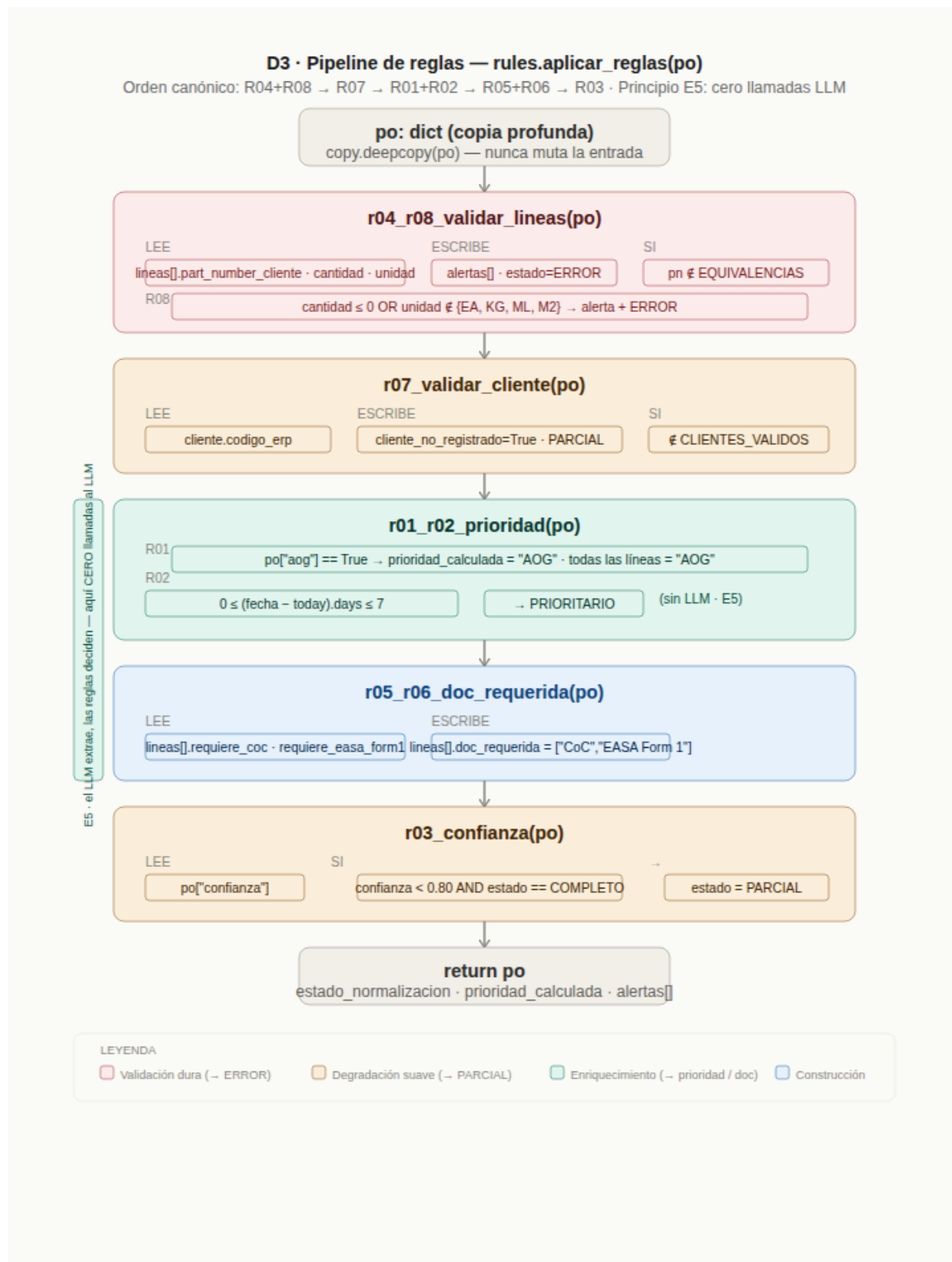
Las ramas 5b, 5c y 5d son conceptualmente paralelas aunque Python las ejecute de forma secuencial. La diferencia importante respecto a n8n es que aquí un fallo en `email_enviar()` lanzaría una excepción que cortaría las ramas que vendrían después. En n8n cada rama tenía su propio `onError: continueErrorOutput`.

Esta diferencia es **deuda técnica consciente**: el Error Trigger global y los reintentos configurables por nodo se implementarán en la sesión II5. En II4 la prioridad es que el flujo correcto funcione; el manejo de errores en los caminos fallidos se aborda en la siguiente sesión.

Idea clave: la deuda técnica no es siempre un problema. En este caso es una decisión de diseño deliberada que permite construir primero el happy path y luego añadir resiliencia de forma controlada.

3. Pipeline de reglas deterministas — R01–R09

El módulo `rules.py` es el corazón del **principio E5** del curso: "el LLM extrae, las reglas deciden". Aquí no hay ninguna llamada al LLM, ninguna conexión de red, ningún efecto de lado. Solo transformaciones puras de un dict Python.



3.1 El orden canónico y por qué importa

La función `aplicar_reglas()` encadena las cinco funciones en un orden específico: `R04+R08 → R07 → R01+R02 → R05+R06 → R03`. Este orden no es arbitrario. Cada función lee campos que las anteriores ya han escrito o validado:

- **R04+R08 primero** — validan los datos de línea antes de intentar validar el cliente. Si hay un part number desconocido, mejor saberlo cuanto antes.
- **R07 segundo** — valida el cliente. Viene después de las líneas porque en teoría podría cancelarse si ya hay error en las líneas (aunque en la implementación actual el flujo continúa en cualquier caso).
- **R01+R02 tercero** — calculan la prioridad. Necesitan que las líneas ya estén validadas para poder leer `fecha_entrega_requerida` con confianza.
- **R05+R06 cuarto** — construyen `doc_requerida` a partir de los flags booleanos. Necesitan que las líneas sean válidas.
- **R03 al final** — degrada el estado si la confianza es baja. Solo tiene sentido ejecutarla al final, cuando el estado ya ha sido determinado por las reglas anteriores.

3.2 Validación dura vs. degradación suave

El diagrama usa dos colores distintos para las dos categorías de consecuencia:

- **Rojo (→ ERROR)** — R04 y R08 marcan `estado_normalizacion = "ERROR"`. La PO no se puede procesar correctamente. En II4 el flujo continúa pero el error queda registrado en `alertas[]`.
- **Ámbar (→ PARCIAL)** — R07 y R03 degradan a "PARCIAL". La PO es procesable pero con advertencias. El operador puede intervenir manualmente.

Esta distinción es importante en producción: un ERROR en R04 significa que el part number del cliente no existe en el catálogo de VERBEX — eso requiere intervención humana urgente. Un PARCIAL por R03 significa simplemente que el LLM no estaba seguro de algún campo — puede resolverse con una revisión rápida.

3.3 El principio E5 en acción

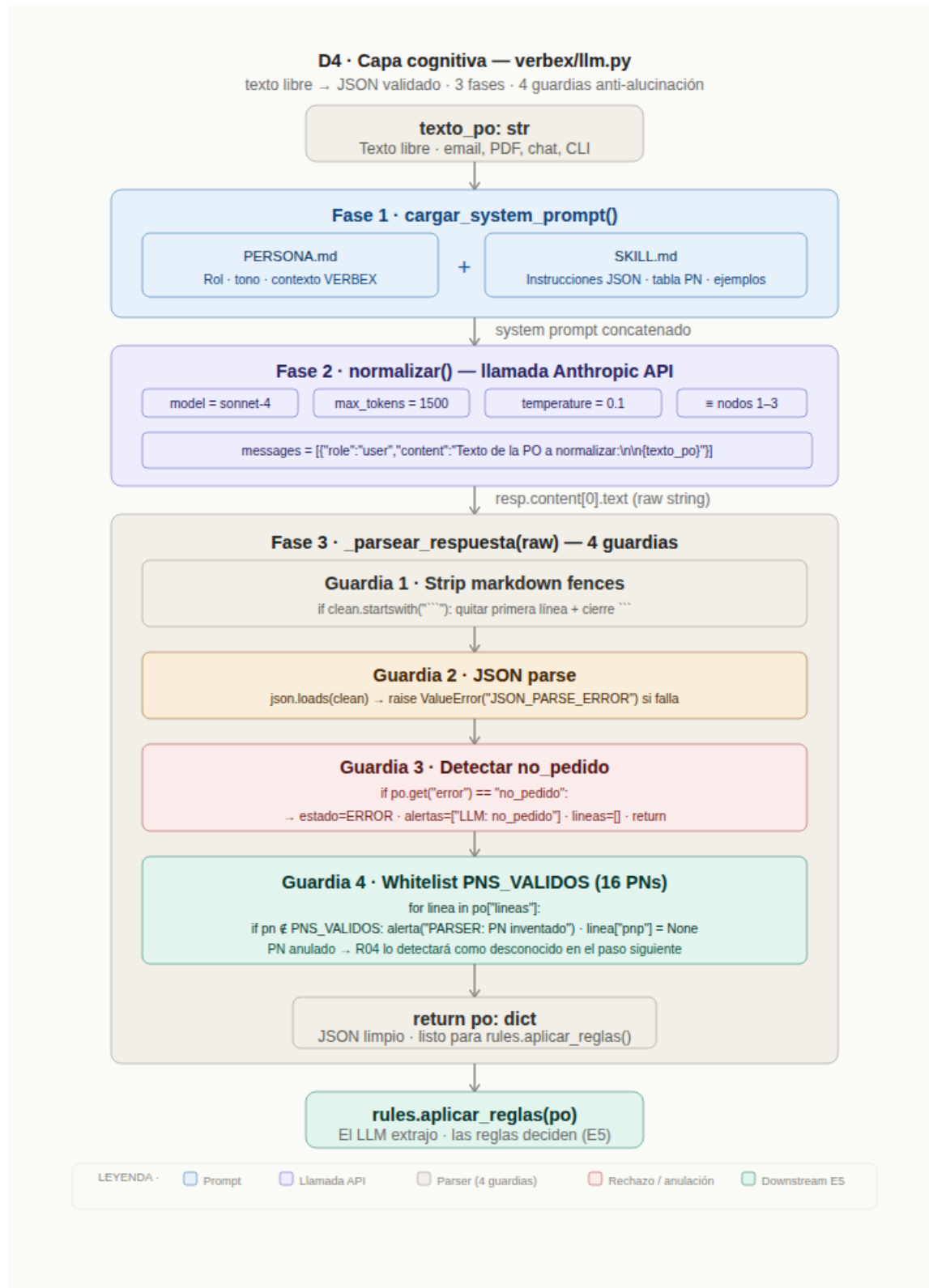
La barra verde en el lateral del diagrama marca todo el bloque de reglas con una anotación: **"E5: el LLM extrae, las reglas deciden — aquí CERO llamadas al LLM"**. Esto no es un detalle de implementación; es el principio arquitectónico más importante del curso.

El antipatrón que E5 evita se llama **E4**: pedir al LLM que calcule valores que son puramente deterministas. El ejemplo canónico del curso es la regla R02: calcular si `(fecha_entrega_requerida - today).days <= 7`. Si esto se delegara al LLM, el nodo completaría sin error pero podría devolver "PRIORITARIO" o "NORMAL" de forma inconsistente dependiendo de cómo el LLM interprete "urgente".

Regla de oro: si puedes escribir la condición como una expresión Python con operadores comparativos (`<`, `>`, `==`, `in`), no necesitas el LLM. Si necesitas juicio semántico ("¿esto parece urgente en contexto?"), ahí sí tiene sentido el LLM.

4. La capa cognitiva — llm.py

El módulo `verbex/llm.py` equivale a los tres primeros nodos del workflow n8n: construcción del body, llamada a la API de Claude, y validación del JSON devuelto. En Python estas tres responsabilidades viven en tres funciones, y la más importante — aunque la menos visible — es el parser anti-alucinación.



D4 — Capa cognitiva llm.py: texto libre a JSON validado

4.1 Fase 1 — Construcción del system prompt

La función `cargar_system_prompt()` lee y concatena `PERSONA.md` y `SKILL.md` en tiempo de ejecución. En n8n estos textos estaban pegados directamente dentro del nodo Claude API; en Python están en ficheros separados y versionables.

La ventaja práctica es la trazabilidad: si el comportamiento del agente cambia, se puede hacer `git diff PERSONA.md` o `git diff SKILL.md` para ver exactamente qué cambió en las instrucciones. Con el texto hardcodeado en el nodo n8n esa traza no existe.

4.2 Fase 2 — Llamada a la API

La función `normalizar()` configura la llamada con tres parámetros clave: `model = "claude-sonnet-4-20250514"`, `max_tokens = 1500`, y `temperature = 0.1`. El valor de temperatura merece atención.

Se usa `0.1` en lugar de `0` por una razón práctica: `temperature = 0` hace la salida completamente determinista, lo que puede causar que el modelo repita exactamente los mismos errores ante las mismas entradas ambiguas. Un valor de `0.1` introduce una pequeña variabilidad que en algunos casos permite al modelo encontrar una interpretación mejor.

4.3 Fase 3 — Las 4 guardias del parser anti-alucinación

La función `_parsear_respuesta()` es la más crítica del módulo. Implementa cuatro guardias en secuencia, cada una protegiendo contra un fallo diferente:

6. **Strip de fences markdown.** Aunque el system prompt pide JSON puro, el LLM a veces envuelve la respuesta en ````json ... ````. La guardia 1 elimina estos delimitadores antes de intentar parsear.
7. **JSON parse.** Intenta `json.loads(clean)`. Si falla, lanza `ValueError("JSON_PARSE_ERROR")`. En n8n este fallo producía un error visible en la UI; en Python se propaga como excepción y puede capturarse con `try/except`.
8. **Detección de no_pedido.** Si el LLM determina que el texto no es una PO (por ejemplo, un mensaje de chat genérico), devuelve `{"error": "no_pedido", ...}`. La guardia 3 normaliza esa señal a `estado_normalizacion = "ERROR"` con una alerta descriptiva.
9. **Whitelist PNS_VALIDOS.** La guardia más sutil. El LLM puede inventarse part numbers de proveedor que suenan plausibles (p. ej. `VBX-COMP-4471-C` en lugar de `VBX-COMP-4471-B`). La guardia 4 verifica cada PN proveedor contra una lista blanca de 16 PNs válidos. Si no está en la lista, el campo se anula con `None` y se añade una alerta.

¿Por qué anular el PN en lugar de rechazar toda la PO? Porque anular permite que el pedido llegue a R04 con el campo a `None`, que R04 lo marque como ERROR, y que el operador vea exactamente qué línea falló. Rechazar la PO entera ocultaría esa información y obligaría al cliente a reenviar todo el pedido.

4.4 La frontera E5 en llm.py

Todo lo que sale de `_parsear_respuesta()` es un dict plano. No hay lógica de negocio aquí — el LLM ha extraído la información del texto libre y el parser la ha validado estructuralmente. A partir de ese `return po`, empieza el territorio de `rules.py`: las reglas deterministas que **deciden** qué hacer con lo que el LLM extrajo.

5. Fan-out de notificaciones — notify.py

El módulo `verbex/notify.py` implementa los cuatro canales de salida del agente. El concepto central de este capítulo es la **independencia de condiciones**: el Switch de prioridad y el Filter de certificación evalúan campos distintos del mismo dict `po` sin ninguna relación entre sí.



D5 — Fan-out de notificaciones: 4 canales, 3 condiciones independientes

5.1 Los cuatro canales

Cada canal tiene su propia condición de disparo, su audiencia y su perfil de idempotencia:

- **Telegram producción (Switch).** Se activa si `prioridad_calculada == "AOG"` (mensaje urgente, <5 s) o `"PRIORITARIO"` (mensaje normal). **NORMAL no envía** — decisión de diseño para no saturar al operador de planta con POs rutinarias.
- **Telegram calidad (Filter).** Se activa si cualquier línea tiene `requiere_coc = True` o `requiere_easa_form1 = True`. Completamente independiente de la prioridad.
- **Email cliente.** Se envía siempre que el cliente tenga `email_contacto` definido, independientemente de prioridad y certificación.
- **Sheets (persistencia).** Se escribe siempre en el camino no-duplicado, antes del fan-out de notificaciones.

5.2 La independencia de condiciones

La confusión más común al leer este módulo es asumir que AOG activa automáticamente Calidad, o que NORMAL nunca genera email. La tabla de combinaciones del diagrama desmonta ambas intuiciones:

Caso	TG Prod.	TG Calidad	Email	Sheets
Stratos · NORMAL + CoC	X	✓	✓	✓
Kairos · AOG + EASA Form 1	✓	✓	✓	✓
NORMAL · sin certificación	X	X	✓	✓
Re-ejecución · duplicado R09	X	X	X	X

Tabla 1 — Combinaciones posibles por canal según caso de prueba

El caso Stratos (NORMAL + CoC) es el más instructivo: no activa Telegram producción porque la prioridad es NORMAL, pero sí activa Telegram calidad porque la línea requiere CoC. Un alumno que no haya revisado el código podría asumir que "si no es urgente, Calidad tampoco necesita saber" — la tabla demuestra que eso es incorrecto.

5.3 Idempotencia asimétrica

Los cuatro canales tienen perfiles de idempotencia muy diferentes, y esa diferencia dicta la arquitectura de error handling en II5:

- **Sheets: idempotente.** El lookup R09 lee antes de escribir. Si la PO ya existe con estado COMPLETO, `append_pedido()` no se ejecuta. Un re-run del workflow desde el principio no genera filas duplicadas.
- **Telegram: NO idempotente.** Cada llamada a `_telegram_send()` entrega un mensaje al canal. No hay mecanismo nativo para detectar duplicados. La guardia es el R09 en `main.py`, que corta el flujo antes de llegar a `notify.py`.
- **Email: NO idempotente.** Cada llamada a `email_enviar()` envía un email a la bandeja del cliente. A diferencia de Telegram, ni siquiera el R09 protege completamente: si el workflow falla **después** de la escritura en Sheets y **antes** de

completar el email, una re-ejecución manual podría generar un duplicado en Telegram pero no en Sheets.

Implicación para II5: el retry configurable por nodo que se implementará en la siguiente sesión debe tener en cuenta este perfil. Sheets puede reintentarse sin guardia adicional; Telegram y Email necesitan una guardia externa (lock, flag en Sheets) antes de poder reintentarse de forma segura.

6. Casos de prueba en clase — Gate F4

La sesión II4 define tres casos de prueba que el alumno debe ejecutar en clase para validar que el workflow funciona correctamente. El criterio de éxito se llama **Gate F4**.

6.1 Los tres casos

Caso 1 — Stratos estándar (PO-2025-STRA-0847)

PO de Stratos Systems con una línea, part number `ST9-HTP-RIB-047`, CoC requerido, sin flag AOG. Resultado esperado:

-

Caso 2 — Kairos AOG (con EASA Form 1)

PO de Kairos con flag AOG activo y línea con EASA Form 1 requerido. Este caso prueba el criterio temporal crítico del Gate F4:

- **Telegram producción: mensaje AOG en menos de 5 segundos (criterio único del Gate F4)**

-

Caso 3 — Re-ejecución duplicada

Re-ejecutar el mismo workflow con la misma PO de Stratos (después de haberla procesado en el caso 1). Prueba la idempotencia del R09:

-

6.2 Criterio Gate F4

Workflow JSON	26 nodos · 20 conexiones · 6 terminales · 4 credenciales ✓
AOG → TG producción	Llega en menos de 5 s ✓
CoC / EASA Form 1	Telegram calidad recibe lista de líneas ✓
Persistencia	POs registradas en Pedidos y Lineas ✓
Deduplicación	R09 evita duplicados entre ejecuciones ✓
Email HTML	Llega al cliente con formato branded VERBEX ✓

Tabla 2 — Criterio de cumplimiento Gate F4

El criterio de los 5 segundos para AOG es el único criterio temporal del Gate F4. Implica que Telegram debe ser el primer canal en procesarse en el fan-out — o, en la versión n8n, que las ramas estén genuinamente en paralelo y no bloqueadas por la escritura en Sheets.

7. Hoja de ruta hacia II5

La sesión II4 cierra con un pipeline funcional pero deliberadamente monolítico: un único workflow de 26 nodos en n8n (o una única función `process()` en Python). La sesión II5 refactoriza ese monolito en módulos independientes y añade la capa de resiliencia.

7.1 Lo que queda fuera de scope en II4

- **Sub-workflows:** refactor a master + 4 sub-workflows independientes (II5)
- **Error Trigger global:** escritura en hoja Errores (II5)
- **Retry logic:** configurable por nodo (II5)
- **Alternativa Nanobot:** misma logica en Python con `nanobot` (II5)
- **Streamlit dashboard:** KPIs y filtros (II6)

7.2 Por qué la modularización no puede esperar más

El workflow monolítico de II4 tiene una vulnerabilidad crítica que la tabla de idempotencia asimétrica del capítulo 5 ya dejaba entrever: si el nodo de email falla, n8n puede reintentar el workflow desde el principio, lo que causaría una segunda escritura en Telegram (el operador recibe el AOG dos veces) aunque Sheets ya esté correcto.

La arquitectura de master + sub-workflows de II5 resuelve esto: cada sub-workflow tiene su propio estado de ejecución, su propio manejo de errores, y puede reintentarse de forma independiente sin afectar a los otros canales. Esta es la materialización en n8n del principio de separación de responsabilidades que el diagrama D1 muestra en la arquitectura Python.

"Lo modular se opera; lo monolítico se rehace." — Idea clave II5. Un workflow que hay que reiniciar desde el nodo 1 cada vez que falla el nodo 23 no es operable en producción.

7.3 Checklist pre-II5

Antes de la sesión II5, el alumno debería tener completados los siguientes puntos del Track B:

10. `pytest tests/ -v` pasa con 8 tests en verde sin red ni credenciales
11. Los tres casos de prueba del Gate F4 ejecutan correctamente en el entorno local
12. El fichero `.env` está configurado con credenciales reales (o de sandbox) y no está en el repositorio Git
13. Se ha revisado la estructura de `data/Pedidos.csv` y `data/Lineas.csv` y las cabeceras coinciden con las definidas en `persistence.py`
14. Se ha leído y comprendido la nota de idempotencia asimétrica del capítulo 5 — especialmente la implicación para el retry de email